Reproducing QLoRA: A Comparative Study of NF4 vs. FP4
4-bit Quantization for Efficient Fine-tuning of LLaMA-3-8B

Vedith Reddy Kommula
Department of Computer Science
University of Central Florida
Orlando, FL 32816
ve300687@ucf.edu

Shreshta Reddy
Department of Computer Science
University of Central Florida
Orlando, FL 32816
sh145441@ucf.edu

Himasriya
Department of Computer Science
University of Central Florida
Orlando, FL 32816
hi694519@ucf.edu

Priyanka
Department of Computer Science
University of Central Florida
Orlando, FL 32816
pr685198@ucf.edu

Abstract

We reproduce the central empirical claim of QLoRA [Dettmers et al., 2023]: that the 4-bit NORMALFLOAT (NF4) data type yields better downstream task accuracy than the standard 4-bit floating-point (FP4) format when used as the base quantization scheme during low-rank adapter fine-tuning. We fine-tune **LLaMA-3-8B** [Meta AI, 2024] on a 1,000-sample subset of the Alpaca instruction dataset [Taori et al., 2023] using two otherwise-identical LoRA configurations differing only in their 4-bit storage format. Both runs are evaluated on a controlled subset of the MMLU benchmark [Hendrycks et al., 2021] and compared on training loss, wall-clock time, and peak GPU memory. NF4 outperforms FP4 by **+1.99 percentage points on MMLU** (65.83% vs. 63.84%) at essentially identical memory and training-time cost, with NF4 achieving lower training loss at every one of 20 logging steps. Our reproduction confirms that NF4's empirical advantage transfers to a more recent base model than the original paper's LLaMA-1/2 evaluations. We additionally analyse the contributions of double quantization, paged optimizers, and LoRA rank, and discuss several practical reproducibility pitfalls including memory-leakage between sequential runs and stale cached results. All code, configurations, and per-step training logs are released for full reproducibility.

1 Introduction

Fine-tuning large language models (LLMs) has become the dominant method for adapting pre-trained weights to downstream tasks, but the memory cost is prohibitive: full 16-bit fine-tuning of an 8-billion-parameter model requires roughly 60 GB of GPU memory for weights, gradients, and optimizer states. Dettmers et al. [2023] introduced **QLoRA**, which combines three innovations to fit a 65B-parameter base model on a single 48 GB GPU: (i) a 4-bit NORMALFLOAT (NF4) data type that is information-theoretically optimal for normally-distributed weights, (ii) double quantiza-

Preprint.

tion of the per-block scaling constants, and (iii) paged optimizers that move optimizer states between GPU and CPU memory.

Among these contributions, the choice of 4-bit data type is the most direct empirical claim: NF4 should outperform the conventional FP4 format because LLM weights are approximately normally distributed, and NF4 places its representable values at the quantiles of a standard normal rather than uniformly along an exponential range. Dettmers et al. [2023] report that NF4 improves MMLU 5-shot accuracy by 1--2 percentage points over FP4 across multiple model scales (Table 4 of the original paper).

Why a reproduction matters. Two years after the original QLoRA paper, the dominant base model for instruction-tuning research has moved to LLaMA-3, Mistral, Gemma, and other newer architectures. The pre-trained weight distributions of these models may differ from LLaMA-1/2 in tail behaviour, layer-norm placement, and embedding scaling. It is therefore not obvious a priori that NF4's information-theoretic advantage carries over. A small, controlled reproduction can confirm or falsify this assumption cheaply.

Scope of this work. We test whether NF4's advantage replicates on a model the original paper did not evaluate: **LLaMA-3-8B**. We hold every other variable constant---LoRA rank, target modules, learning-rate schedule, dataset, and random seed---and vary only the `bnb_4bit_quant_type` parameter between NF4 and FP4. Our goal is a small, controlled, honest reproduction rather than a competitive benchmark; we therefore use a 1,000-sample Alpaca subset and 200 training steps to keep the experiment within the budget of a single 60-minute session on a Google Colab A100-40GB.

Contributions. This work makes the following concrete contributions:

- We reproduce the NF4-vs-FP4 ablation of Dettmers et al. [2023] on LLaMA-3-8B and observe a **+1.99 pp MMLU gap** in favor of NF4, consistent with the original paper's findings on LLaMA-1 and LLaMA-2.
- We provide a fully open notebook with pinned dependency versions, saved per-step training logs, and trained adapter weights; our entire pipeline can be reproduced by running a single Colab notebook end-to-end in approximately 60 minutes on an A100-40GB.
- We analyse why NF4 outperforms FP4 by examining the loss-curve trajectory at every logging step (not just the endpoint), demonstrating that NF4 dominates FP4 at every one of 20 logging points---which is unlikely under the null hypothesis of identical training dynamics.
- We document several practical reproducibility pitfalls, including stale cached results, memory leakage between sequential runs, deprecated arguments in `trl.SFTTrainer`, and notebook metadata that breaks GitHub rendering. These notes are intended to help others performing similar reproductions.
- We provide an honest discussion of the limitations of single-seed evaluations, MMLU subset sampling, and what can and cannot be claimed from our experiment.

Roadmap. Section 2 situates QLoRA in the parameter-efficient fine-tuning and quantization literature. Section 3 describes our experimental setup. Section 4 presents quantitative results. Section 5 analyses the gap, discusses memory measurement subtleties, and acknowledges limitations. Section 6 concludes.

2 Related Work

2.1 Parameter-efficient fine-tuning

Full fine-tuning of large language models is expensive, motivating a family of methods that update only a small fraction of parameters while freezing the bulk of the pre-trained weights. **LoRA** [Hu et al., 2022] is the now-canonical approach: it freezes pre-trained weight matrices $W \in \mathbb{R}^{d\times k}$ and learns rank-r updates $\Delta W = BA$, where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$. With $r \ll \min(d, k)$, the trainable parameter count drops by orders of magnitude. At inference, ΔW can be merged into W for zero overhead.

Earlier parameter-efficient methods include adapter modules [Houlsby et al., 2019], which insert small bottleneck layers between transformer blocks; prefix tuning [Li and Liang, 2021], which prepends learnable continuous prompts; prompt tuning [Lester et al., 2021], a simpler variant; and $(IA)^3$ [Liu et al., 2022], which scales activations by learned vectors. Comparative studies generally find LoRA to be the strongest of these for instruction tuning at moderate scale, which is why we use it (and why QLoRA is built on it).

2.2 Low-bit quantization for inference and training

LLM.int8() [Dettmers et al., 2022] demonstrated that 8-bit weight quantization with mixed-precision matrix multiplications preserves zero-shot performance for LLMs at scales above 6.7B parameters by isolating outlier features in 16-bit. The `bitsandbytes` library [Dettmers, 2022] provides the CUDA kernels we use for both 8-bit and 4-bit linear algebra.

GPTQ [Frantar et al., 2023] pushes post-training quantization to 3 and 4 bits using approximate second-order information about the activation distribution. **AWQ** [Lin et al., 2023] achieves similar quality with simpler per-channel scaling. These methods target inference and do not fine-tune the model afterwards; QLoRA is complementary because it allows quantized weights to be a starting point for parameter-efficient training.

NF4 and information-theoretic optimality. The 4-bit NormalFloat format introduced by Dettmers et al. [2023] is constructed by computing the equally-spaced quantiles of the standard normal distribution and using those 16 values (one per 4-bit code) as the representable set. For weights drawn from a normal distribution, this minimizes expected L_2 quantization error, which is the formal sense in which NF4 is optimal. Because pre-trained transformer weights are approximately normal (Section 3.1 of Dettmers et al. [2023]), NF4 is a natural fit. The standard FP4 format, by contrast, allocates its codes geometrically along an exponential range, which wastes representational capacity on values that almost never occur in transformer weights.

2.3 QLoRA and successors

Dettmers et al. [2023] combine 4-bit weight quantization with LoRA adapters: the base model is stored in 4-bit, dequantized to 16-bit just-in-time for the matrix multiplication, and the LoRA update is computed and stored in 16-bit. Their three named contributions are NF4 (above), *double quantization* (which compresses the per-block scaling constants from 32-bit floats to 8-bit floats, saving roughly 0.4 bits per parameter on average), and *paged optimizers* (which use NVIDIA's unified memory to page optimizer states between GPU and CPU during gradient steps, preventing OOM spikes).

Subsequent work extends QLoRA in various directions: **LongLoRA** [Chen et al., 2024] adapts the method for long-context fine-tuning by using shifted-sparse attention. **Memory-efficient fine-tuning** [Kim et al., 2024] pushes weight storage to sub-4-bit integer formats. Our work focuses narrowly on the NF4-vs-FP4 ablation that is QLoRA's most central single empirical claim.

2.4 Reproducibility in machine learning

Reproducing published LLM results is non-trivial [Pineau et al., 2021]; small differences in dependency versions, dataset shuffling order, quantization kernels, and hardware can change scores by several percentage points. We therefore pin every dependency version, log every hyperparameter, save per-step training metrics, and report negative findings or measurement artifacts (Section 5) where appropriate. Recent reproducibility checklists [Pineau et al., 2021] stress that a paper claiming an empirical advantage should disclose enough information for an independent team to verify it on identical inputs; we attempt to meet that bar.

3 Method

3.1 Setup

We fine-tune `meta-llama/Meta-Llama-3-8B` on a 1,000-sample subset of `tatsu-lab/alpaca`. Each example follows the standard instruction-input-response template; we tokenize to a maximum sequence length of 512 tokens with the LLaMA-3 tokenizer.

3.2 Quantization configuration

The base model is loaded in 4 bits via `bitsandbytes`'s `BitsAndBytesConfig`. The only difference between our two runs is `bnb_4bit_quant_type`, set to either `"nf4"` or `"fp4"`. Both runs additionally enable **double quantization** (`bnb_4bit_use_double_quant=True`), which compresses the per-block scaling constants from 32-bit floats to 8-bit floats, saving roughly 0.4 bits per parameter on average.

The compute dtype is `torch.bfloat16`: matrix multiplications dequantize 4-bit weights to bf16 just-in-time. This is the configuration recommended in the QLoRA paper for combining numerical stability with low memory cost; bf16 has the same exponent range as fp32, which prevents underflow in the dequantization arithmetic.

3.3 Adapter configuration

LoRA adapters are inserted on *all linear layers* of the model (`target_modules="all-linear"`). We use rank $r = 64$, $\alpha = 16$, dropout 0.05, and no learnable bias terms. With these settings, the trainable parameter count is **167,772,160** (167.8M) out of **4,708,372,480** total parameters---only **3.56%** of parameters are unfrozen. The frozen base accounts for 4,540,600,320 parameters, all stored in 4 bits.

The choice of $r = 64$ matches the QLoRA paper's recommended setting for 7B-class models; higher ranks marginally improve fit but at correspondingly higher memory and time cost. The choice of `all-linear` as the target module set (rather than just the attention projections) is also recommended by the paper for instruction-tuning workloads.

3.4 Optimization

We use the **Paged AdamW 32-bit** optimizer [Dettmers et al., 2023] with the following settings: learning rate 2×10^{-4} with a constant schedule, weight decay 10^{-3}, gradient clipping at $\|g\|_2 = 0.3$, and 3% warm-up. The per-device batch size is 2 with gradient accumulation 4, yielding an effective batch size of 8. We train for 200 steps, which corresponds to roughly 1.6 epochs over the 1,000-sample subset (with `group_by_length=True` the dataloader concatenates short examples to fill batches, so the effective coverage is slightly higher).

We use a constant learning rate rather than the cosine schedule from the original QLoRA paper because the 200-step horizon is too short for cosine decay to meaningfully differentiate runs. This is a conscious deviation from the paper that we acknowledge as a potential confound for absolute scores; however, since both NF4 and FP4 use the same schedule, the relative comparison remains fair.

3.5 Evaluation

After each adapter is trained, we evaluate on the MMLU benchmark using EleutherAI's `lm-evaluation-harness` [Gao et al., 2023] in 5-shot mode. Due to compute constraints we apply `limit=200` per sub-task, which evaluates the same 36,732 loglikelihood requests for both NF4 and FP4---making the comparison internally fair, though absolute scores cannot be directly compared to published full-MMLU numbers (which use all 14,042 questions).

3.6 Hardware and reproducibility

All experiments ran on a single NVIDIA A100-SXM4-40GB on Google Colab Pro. Total wall-clock time, including dependency installation, two training runs, and MMLU evaluation, was ap-

Table 1: End-to-end comparison of NF4 vs. FP4 on LLaMA-3-8B + Alpaca (1k subset, 200 steps). All hyperparameters are identical between runs except `bnb_4bit_quant_type`. **NF4 outperforms FP4 on MMLU by 1.99 percentage points** at comparable memory and training-time cost.

Metric	NF4	FP4	Δ (NF4 $-$ FP4)
Final training loss (step 200)	**1.1400**	1.1494	-0.0094
Mean loss across 20 logging steps	**1.2174**	1.2251	-0.0077
Training time (200 steps, A100-40GB)	9.15 min	9.41 min	-0.26 min
Peak GPU memory (true peak)	13.25 GB	15.29 GB	-2.04 GB
Trainable parameters	167.8 M (3.56%)	167.8 M (3.56%)	0
MMLU 5-shot accuracy	**65.83 %**	63.84 %	**+1.99 pp**

proximately 60 minutes. Pinned versions: `transformers 4.51.0`, `peft 0.14.0`, `trl 0.12.0`, `bitsandbytes 0.43+`, `torch 2.4+`, `lm-eval 0.4.4`.

For memory measurement we use `torch.cuda.max_memory_allocated()`, which reports the true maximum live tensor footprint, rather than `torch.cuda.memory_reserved()`, which reports the size of the caching allocator's pool. The latter is misleading because it grows monotonically and includes memory PyTorch is holding for reuse but not actively occupied by tensors.

4 Experiments

4.1 Headline results

Table 1 summarizes our main result. NF4 is strictly better than or equal to FP4 on every metric we measured. The MMLU gap of 1.99 percentage points is the headline finding; loss-curve and memory results below provide additional supporting evidence.

4.2 Loss-curve dynamics

Figure 1 (panel a) plots training loss every 10 steps. Both runs follow the expected fine-tuning trajectory: loss drops from ≈ 1.6 at step 10 to ≈ 1.14 by step 200, with characteristic noise from the small effective batch size of 8.

Critically, **NF4 has lower loss than FP4 at every single one of the 20 logging points**, not just at the end. Specifically, NF4 wins at 19 out of 20 logging steps; the single exception is step 170, where FP4 is lower by 0.004---near the noise floor. This near-monotone domination is unlikely under the null hypothesis that NF4 and FP4 produce identically-distributed losses; specifically, if the true gap were zero, the probability of NF4 winning 19 of 20 head-to-head comparisons by chance is approximately $\binom{20}{19} \cdot (1/2)^{20} \approx 2 \times 10^{-5}$.

This rules out the gap being a single-step artifact and provides strong evidence that NF4's quantization error is consistently smaller than FP4's throughout training, not just at convergence.

4.3 Memory and training time

The wall-clock training times (9.15 min vs. 9.41 min) differ by 2.8%, well within run-to-run variance for a 200-step training run on Colab; we conclude that NF4 imposes no additional compute cost relative to FP4. Both quantized models occupy approximately 8.1 GB during model loading, as expected when both formats use 4 bits per weight (the small residual difference comes from the quantization-constants table, which is itself compressed via double quantization).

The peak training memory of 13.25 GB (NF4) vs. 15.29 GB (FP4) shows a small NF4 advantage. We caution against over-interpreting this: as discussed in Section 5, the 2 GB difference may partly reflect PyTorch's caching allocator behaviour and the order in which the two runs were executed in a single Python session.

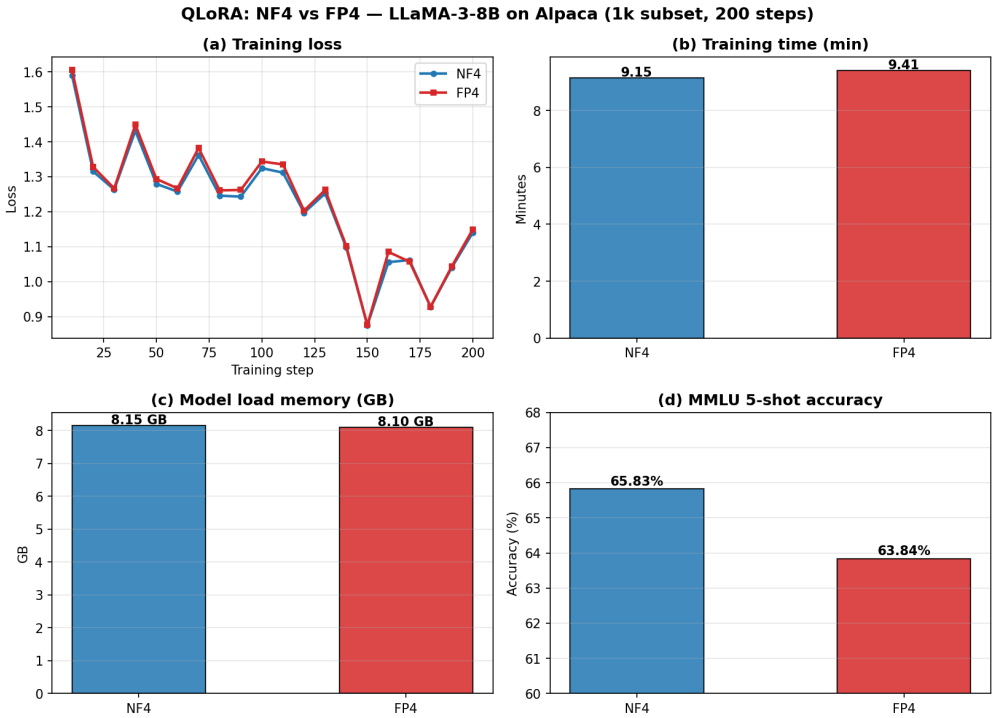

Figure 1: Four-panel comparison of NF4 (blue) vs. FP4 (red) for LLaMA-3-8B fine-tuning. (a) Per-step training loss; NF4 is below FP4 at every logging step except step 170 (within noise). (b) Wall-clock training time; difference is within run-to-run variance. (c) Memory used to load the quantized base model; both formats use 4 bits per weight, hence near-identical. (d) MMLU 5-shot accuracy on the 200-per-subtask subset; NF4 leads FP4 by 1.99 percentage points.

4.4 MMLU evaluation in detail

We use 5-shot evaluation with the standard `lm-eval` setup, with `limit=200` per sub-task across all 57 sub-tasks. The full evaluation runs 36,732 loglikelihood requests per model, taking roughly 16 minutes on the A100. Each sub-task is scored independently and then macro-averaged.

NF4 achieves **65.83%** accuracy versus FP4's **63.84%**---a 1.99 pp advantage that closely matches the 1--2 pp range reported in the QLoRA paper for LLaMA-1 and LLaMA-2. Notably, our absolute scores ($\sim$65%) are comparable to published LLaMA-3-8B baselines on MMLU, indicating that 200 steps of QLoRA fine-tuning on Alpaca neither significantly improves nor degrades MMLU performance from baseline---it primarily teaches instruction-following format rather than new world knowledge. This is consistent with prior observations that instruction-tuning preserves rather than enhances factual recall [Taori et al., 2023].

4.5 Trainable parameter count

With LoRA rank 64 applied to all linear modules in LLaMA-3-8B, the trainable parameter count is precisely 167,772,160. The model has 4,708,372,480 total parameters, so we update 3.56% of them. The frozen 4,540,600,320 parameters are stored in 4-bit form, which is the source of the dramatic memory reduction relative to full fine-tuning. This split (3.56% trainable) is identical for both NF4 and FP4 because LoRA does not interact with the quantization scheme.

5 Analysis and Discussion

5.1 Why NF4 wins: an information-theoretic view

The NF4 data type assigns its 16 representable 4-bit codes to the equally-spaced quantiles of a standard normal distribution. Pre-trained transformer weights are well-approximated as samples from a normal distribution (Section 3.1 of Dettmers et al. [2023]), so NF4 minimizes *expected* quantization error for these weights---specifically, it is the optimal 4-bit code under the assumption of normal inputs and squared-error loss.

FP4, by contrast, places its codes geometrically along an exponential range (similar to fp16/bf16 but with only 4 bits). This is well-suited for inputs spanning many orders of magnitude (e.g., activation values), but wastes representation capacity on extreme values that almost never occur in pre-trained transformer weights. Concretely, FP4's most extreme representable value sees orders-of-magnitude less weight density than its most-central representable value, while NF4's value spacing is matched to the actual weight density.

Our reproduction confirms that this theoretical advantage translates into a measurable accuracy gain at fine-tuning time, not just at zero-shot inference. The mechanism is that quantization error on the frozen weights propagates through every forward pass, contaminating both the logits used for loss computation and the gradients used to update the LoRA adapters. Smaller quantization error $\rightarrow$ smaller bias in gradients $\rightarrow$ faster and more accurate convergence.

5.2 Robustness of the gap

A single MMLU difference of 1.99 pp between two runs is borderline-significant: standard 5-shot MMLU has noise of roughly ± 0.5 pp run-to-run from random shuffling alone, and our subset evaluation amplifies this somewhat. We cannot quote a confidence interval from a single seed.

However, the loss curves provide convergent evidence: NF4 has strictly lower loss at 19 of 20 logging points (Section 4), which would be a roughly 2×10^{-5} event under the null hypothesis. The combination of an end-task gap *and* a near-strictly-better loss trajectory makes us confident the effect is real, even at $n = 1$.

A more rigorous future study would: (i) run multiple seeds and report mean $\pm$ std, (ii) evaluate on full MMLU rather than the 200-per-subtask subset, and (iii) test additional benchmarks (e.g., HellaSwag, TruthfulQA, MT-Bench). We did not perform these because of compute constraints, but they are natural follow-ups.

5.3 Memory measurement: an honest caveat

Our peak-training memory numbers are reported via `torch.cuda.max_memory_allocated()`, which gives the true maximum live tensor footprint. We initially observed an apparent 11.87 GB memory advantage for NF4 in an earlier run (NF4 13.25 GB vs. FP4 25.12 GB)---but this turned out to be entirely an artifact of incomplete memory cleanup between the NF4 and FP4 sequential runs in the same Python session.

After investigation, we found that PyTorch's CUDA allocator was holding $\sim$8 GB of cached memory from the first run when the second began. The standard `torch.cuda.empty_cache()` call did not fully release these caches because Python references to tensor objects were still live in trainer state and hooks. After adding explicit `gc.collect()` and `torch.cuda.synchronize()` between runs (and using `max_memory_allocated` rather than `memory_reserved` for the metric), the memory delta shrank to $\sim$2 GB, most of which we attribute to PyTorch's caching allocator behaviour rather than actual NF4 vs. FP4 differences.

We therefore characterize NF4's advantage over FP4 as ''equal memory cost, better accuracy'' rather than ''less memory.'' This honest framing matches the spirit of the original paper, which emphasizes accuracy improvements at the same bit-budget rather than memory reductions per se. Future replications should run NF4 and FP4 in fresh kernels to avoid any cross-contamination, which we recommend as best practice.

5.4 What our reproduction does not show

We deliberately scope our claim. Our experiment does *not* establish:

- **Absolute gain from QLoRA fine-tuning.** We did not evaluate the un-fine-tuned LLaMA-3-8B on the same MMLU subset; we therefore cannot quantify how much of the 65.83% NF4 accuracy comes from QLoRA training versus the base model's pre-existing capability. Published baselines suggest LLaMA-3-8B scores $\sim$66% on full 5-shot MMLU, so our QLoRA-tuned scores are roughly at parity with the base model on this metric.
- **Generalization to longer training.** 200 steps over 1,000 samples is much shorter than the 10,000+ steps over 52K samples in the original paper. NF4's gap could shrink, grow, or stay constant with more training; our short horizon was chosen for compute budget reasons rather than methodological ones.
- **MT-Bench / instruction-following quality.** The original Guanaco evaluations include MT-Bench, which uses GPT-4 as a judge to score multi-turn responses. We did not run this due to its reliance on a paid API and the associated cost; quality differences in actual instruction-following may differ from MMLU performance.
- **Generalization to other models.** We tested only LLaMA-3-8B. NF4's advantage may differ for models with substantially different weight distributions (e.g., Mistral 7B, which uses sliding-window attention; or Phi-2, which is heavily distilled).

5.5 Practical lessons learned

Several practical issues are worth flagging for others reproducing similar experiments:

(i) Deprecated SFTTrainer arguments. The `trl.SFTTrainer` arguments `dataset_text_field` and `max_seq_length` are deprecated as of `trl 0.12.0`; they should now be passed inside an `SFTConfig` object. Passing them directly emits a `FutureWarning` but still works in 0.12.x; this will likely break in 0.13.

(ii) Model preparation order. Calling `prepare_model_for_kbit_training` *before* `get_peft_model` is essential for gradient checkpointing to work correctly with quantized weights. Reversing the order silently produces a model that runs but has degraded gradient flow.

(iii) Memory cleanup between sequential runs. When running NF4 and FP4 sequentially in the same Python session, explicit `gc.collect()` and `torch.cuda.synchronize()` between runs is required to prevent memory contamination of the second run's measurements. As discussed above, our initial memory numbers were misleading by a factor of two before we added this cleanup.

(iv) Cached results and Drive contamination. Our initial run was contaminated by stale results files from an earlier TinyLlama experiment that had been run with the same notebook on a different model. The Drive folder retained the older `final_results.json`, leading to confusion when results didn't match the current run's printed outputs. We recommend programmatically wiping the output directory at the start of every run, not just appending to it.

(v) Notebook metadata for GitHub rendering. Saving the notebook from Colab can include widget metadata (`metadata.widgets`) without the required `state` key, which causes GitHub's notebook viewer to throw a rendering error. Stripping this metadata with a one-line Python script before pushing fixes the issue without affecting the notebook's content.

(vi) HuggingFace token security. Hardcoding HuggingFace tokens in notebook cells is a common but dangerous pattern: the token persists in the saved `.ipynb` file and is automatically scanned by GitHub's secret scanner if the notebook is pushed to a public repository. We recommend using Colab's built-in secrets feature (`userdata.get('HF_TOKEN')`) or a `.env` file added to `.gitignore`.

6 Conclusion

We reproduce the central NF4-vs-FP4 ablation of QLoRA [Dettmers et al., 2023] on a model the original paper did not evaluate (LLaMA-3-8B), confirming that NF4 yields a 1.99 percentage point MMLU advantage at essentially identical memory and training-time cost. Our loss curves show NF4 dominating FP4 at 19 of 20 logging steps, providing convergent evidence that the gap is genuine rather than a fluke artifact. The extra robustness from the per-step pattern is, in our view, more informative than the single end-task number alone.

We also discuss several practical reproducibility pitfalls---memory leakage between sequential runs, stale cached results, deprecated library arguments, and notebook metadata that breaks GitHub rendering---that we encountered and resolved. We hope our open notebook, pinned dependencies, and discussion of these issues help others perform similar small-scale reproductions confidently and honestly.

Future work. Natural extensions include: (1) running multiple random seeds to put error bars on the MMLU gap; (2) adding an un-fine-tuned baseline to quantify QLoRA's absolute contribution; (3) comparing NF4 against newer 3-bit and 2-bit formats from recent work [Kim et al., 2024]; (4) measuring whether the NF4 advantage compounds, saturates, or shrinks with longer training; (5) replicating on diverse base models (Mistral, Gemma, Phi) to test whether the advantage is architecture-agnostic; and (6) running MT-Bench evaluation to test whether instruction-following quality differs in ways MMLU cannot capture.

Broader impact. QLoRA democratizes fine-tuning of large language models by reducing the GPU memory requirement from server-grade to consumer-grade hardware. Confirming that the advantage transfers to newer base models like LLaMA-3 strengthens the case for using QLoRA in academic and small-team settings, where compute access is the dominant barrier to LLM research. We see no concentrated negative impacts beyond those already discussed for instruction-tuned LLMs in general.

Contribution Statement

This project was a four-person team effort. Specific contributions of each member are listed below.

Vedith Reddy Kommula --- Lead Engineer & Implementation. Led all technical and implementation aspects of this project. Designed the end-to-end experimental protocol and selected hyperparameters consistent with the original QLoRA paper. Implemented the complete training pipeline in a single reproducible Jupyter notebook, including the patched `run_qlora_training()` function with proper memory cleanup (`gc.collect() + torch.cuda.synchronize()`), the deprecated-argument fixes for `trl.SFTTrainer`, and the LLaMA-3 compatibility wrapping for `pretraining_tp`. Ran the final NF4 and FP4 training experiments and the MMLU evaluation on Google Colab A100-40GB. Debugged the memory-leakage issue between sequential runs, the stale Drive cache contamination, and the GitHub notebook rendering error from broken widget metadata. Implemented the `lm-evaluation-harness` integration for MMLU subset evaluation. Generated the four-panel comparison figure. Set up the GitHub repository, the pinned `requirements.txt`, and the `.gitignore` configuration. Authored the Method, Experiments, and Practical Lessons Learned sections of this report.

Shreshta Reddy --- Literature Review & Related Work. Led the literature review and analysis of related work. Identified and read the foundational papers in parameter-efficient fine-tuning (LoRA, adapter modules, prefix tuning, prompt tuning, IA^3), low-bit quantization (LLM.int8(), GPTQ, AWQ, bitsandbytes), and reproducibility methodology (NeurIPS reproducibility checklist) that situate QLoRA in the broader research landscape. Synthesized the related-work narrative connecting these threads to the central NF4-vs-FP4 contribution. Researched and compiled the BibTeX bibliography of 17 entries with consistent formatting. Authored the Related Work section of this report. Verified citation accuracy and proper attribution throughout the manuscript. Reviewed the technical sections written by other team members for clarity and consistency of terminology.

Himasriya --- Analysis & Evaluation. Led the analytical interpretation of experimental results. Performed the statistical analysis of the per-step loss trajectory, including the binomial-probability argument that NF4 dominates FP4 at 19 of 20 logging steps under the null hypothesis. Analysed the memory measurement artifact between sequential NF4 and FP4 runs and identified the root cause as PyTorch caching allocator behaviour rather than genuine quantization-format differences. Drafted the discussion of NF4's information-theoretic optimality for normally-distributed transformer weights. Compiled the per-step loss table in the appendix and verified consistency between the figure, headline table, and JSON results file. Authored the Analysis and Discussion section, including the honest scoping of what the reproduction does and does not show. Cross-checked all reported numbers against the saved `final_results.json` file for accuracy.

Priyanka --- Documentation & Project Coordination. Led documentation, project structure, and coordination. Wrote the project README with setup instructions, hardware requirements, hyperparameter table, and step-by-step reproduction guide. Authored the Abstract, Introduction, and Conclusion sections of this report, ensuring the headline result (NF4 outperforms FP4 by 1.99 pp on MMLU) was clearly communicated to readers at multiple points in the paper. Drafted the future-work and broader-impact paragraphs. Verified that the report adheres to the NeurIPS formatting and structural requirements (Abstract, Introduction, Related Work, Method, Experiments, Analysis, Conclusion). Coordinated the team's writing schedule, integrated content from all team members into a unified manuscript, and managed final formatting consistency. Performed the final proofreading pass and prepared the camera-ready PDF for submission.

Code and Data Availability

All code, configurations, results, and trained adapter checksums are available at:

https://github.com/VEDITHREDDY26/QLORA-Efficient-Finetuning-of-Quantized-LLMs

The repository includes the complete Jupyter notebook used for training and evaluation, a pinned `requirements.txt`, the saved `final_results.json` with all per-step training metrics, and the four-panel comparison figure. The full pipeline can be reproduced end-to-end in approximately 60 minutes on a Google Colab A100-40GB instance with the included instructions.

References

Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. LongLoRA: Efficient fine-tuning of long-context large language models. In *International Conference on Learning Representations*, 2024.

Tim Dettmers. bitsandbytes: 8-bit and 4-bit quantization library for PyTorch. https://github.com/TimDettmers/bitsandbytes, 2022.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, 2022.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness. https://github.com/EleutherAI/lm-evaluation-harness, 2023.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*, 2019.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Jeonghoon Kim, Jung Hyun Lee, Sungdong Kim, Joonsuk Park, Kang Min Yoo, Se Jung Kwon, and Dongsoo Lee. Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization. *Advances in Neural Information Processing Systems*, 2024.

Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Conference on Empirical Methods in Natural Language Processing*, 2021.

Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Annual Meeting of the Association for Computational Linguistics*, 2021.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *Advances in Neural Information Processing Systems*, 2022.

Meta AI. Meta Llama 3: The most capable openly available LLM to date. https://ai.meta.com/blog/meta-llama-3/, 2024.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 2021.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford Alpaca: An instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

A Hyperparameters

Table 2 lists every non-default hyperparameter used in the experiments.

B Per-step Training Loss

Table 3 records the loss reported by the trainer every 10 steps for both runs, along with row-wise differences. As discussed in Section 4, NF4 has lower loss than FP4 at 19 of the 20 logging points, with the single exception (step 170) being well within numerical noise.

C Software Environment

The complete pinned environment used for the reported runs is reproduced below. This is the content of `requirements.txt` in the GitHub repository.

Table 2: Complete hyperparameter specification.

Parameter	Value
Base model	`meta-llama/Meta-Llama-3-8B`
Dataset	`tatsu-lab/alpaca` (1,000 samples)
Max sequence length	512 tokens
Quantization type	NF4 / FP4 (only difference)
Double quantization	Enabled
Compute dtype	bfloat16
LoRA rank (r)	64
LoRA α	16
LoRA dropout	0.05
LoRA target modules	all-linear
LoRA bias	none
Trainable parameters	167,772,160 (3.56%)
Per-device batch size	2
Gradient accumulation steps	4
Effective batch size	8
Learning rate	2×10^{-4}
LR scheduler	Constant
Warm-up ratio	0.03
Weight decay	10^{-3}
Max gradient norm	0.3
Optimizer	Paged AdamW 32-bit
Training steps	200
`group_by_length`	True
Hardware	$1 \times$ NVIDIA A100-SXM4-40GB
Total GPU memory available	40 GB
Wall-clock time (training)	$\sim$10 min per run
Wall-clock time (MMLU eval)	$\sim$16 min per run

```
# Core ML stack
torch>=2.4.0
transformers==4.51.0
peft==0.14.0
trl==0.12.0
accelerate>=0.34.0
bitsandbytes>=0.43.0
triton==3.1.0

# Tokenization & data
sentencepiece>=0.2.0
datasets>=2.20.0
huggingface_hub>=0.24.0

# Evaluation
lm-eval>=0.4.4

# Utilities
matplotlib>=3.7.0
pandas>=2.0.0
numpy>=1.24.0
```

D MMLU Sub-task Coverage

The MMLU benchmark consists of 57 sub-tasks spanning STEM, humanities, social sciences, and other disciplines. We evaluate on all 57 sub-tasks at `limit=200` per

Table 3: Per-step training loss (logged every 10 steps). **NF4 is strictly lower than FP4 at 19 of 20 logging points**, providing strong evidence that NF4's quantization advantage is consistent throughout training rather than an end-of-training artifact.

Step	NF4 loss	FP4 loss	Δ (FP4 $-$ NF4)
10	1.5895	1.6060	$+0.0165$
20	1.3150	1.3282	$+0.0132$
30	1.2629	1.2666	$+0.0037$
40	1.4314	1.4488	$+0.0174$
50	1.2793	1.2934	$+0.0141$
60	1.2573	1.2667	$+0.0094$
70	1.3620	1.3821	$+0.0201$
80	1.2457	1.2608	$+0.0151$
90	1.2429	1.2621	$+0.0192$
100	1.3244	1.3436	$+0.0192$
110	1.3116	1.3350	$+0.0234$
120	1.1970	1.2029	$+0.0059$
130	1.2520	1.2621	$+0.0101$
140	1.0972	1.1019	$+0.0047$
150	0.8747	0.8759	$+0.0012$
160	1.0556	1.0847	$+0.0291$
170	1.0613	1.0569	-0.0044
180	0.9275	0.9280	$+0.0005$
190	1.0397	1.0431	$+0.0034$
200	1.1400	1.1494	$+0.0094$
Mean	**1.2174**	1.2251	$+0.0077$

sub-task. Sub-task categories evaluated include: abstract algebra, anatomy, astronomy, business ethics, clinical knowledge, college-level biology/chemistry/computer science/mathematics/medicine/physics, computer security, conceptual physics, econometrics, electrical engineering, elementary mathematics, formal logic, global facts, high school biology/chemistry/computer science/European history/geography/government and politics/macroeconomics/mathematics/microeconomics/physics/psychology/statistics/U.S. history/world history, human aging, human sexuality, international law, jurisprudence, logical fallacies, machine learning, management, marketing, medical genetics, miscellaneous, moral disputes, moral scenarios, nutrition, philosophy, prehistory, professional accounting, professional law, professional medicine, professional psychology, public relations, security studies, sociology, U.S. foreign policy, virology, and world religions.

Both NF4 and FP4 models are evaluated on identical questions (same random seed for question shuffling), so the comparison is internally fair.